重邮802数据结构押题卷（三）参考答案

对应临时区原《押题卷三.pdf》。按原题号作答；树根层次为1，KMP按1基下标、next[1]=0。涉及图示、代码印刷错误及条件不足之处，均在题下说明。

一、程序阅读题

1．BST非递归查找（5分）

按题意，各空依次为 **BST->data；lchild；rchild；false**。

原题比较行写为 if(item=____)，此处应是相等比较：完整条件须改为 if (item == BST->data)；若严格只在原单个等号后的空处填写，应填 = BST->data，从而组成双等号。while行的印刷错误也须统一为 while (BST != NULL)。

关键字相等即成功，小于当前根则查左子树，大于则查右子树。时间O(h)，辅助空间O(1)，h为树高。

2．单链表操作（5分）

（1）S1沿next向后扫描，使p指向原链表尾结点。

（2）S2把原首结点q接到尾部，再把其next置空，使其成为新尾结点。

（3）n≥2时返回 **(a2,a3,...,an,a1)**，相当于循环左移一位；空表或单结点表保持原状。时间O(n)，辅助空间O(1)。

3．BST查找填空（5分）

两空依次为：

```
return t
t = t->rchild
```

题面空后已有分号。循环可能因t变空而结束，原函数漏写查找失败返回值，应在while之后、函数结束之前补 return NULL;。否则非空树中的查找失败会落到非void函数末尾。字段和运算符中的多余空格按正常C语法恢复。

4．冒泡排序填空（5分）

两空依次为 **n-i；r[j+1]=r[j]**。

原题还须改正：n-1为n-1；if后的左圆括号应为左花括号；提前结束条件 exchange=-0 应为 exchange==0。修正后的主要循环如下，数组下标为0至n-1。

```
for (int i = 1; i <= n - 1; ++i) {
    int exchange = 0;
    for (int j = 0; j < n - i; ++j) {
        if (r[j] > r[j + 1]) {
            int temp = r[j + 1];
            r[j + 1] = r[j];
            r[j] = temp;
            exchange = 1;
        }
    }
    if (exchange == 0) return;
}
```

最终按非递减次序排序；最坏和平均时间O(n^2)，已有序时O(n)，辅助空间O(1)。

二、填空题

1．先序和森林棵数（3分）

E,A,C,B,D,G,F；2棵。后序末项E为根，左子树先序A,C,B,D，右子树先序G,F；对应森林的根沿原二叉树右孩子链E→G排列，共2棵。

2．表达式中的栈（3分）

运算符及左括号；操作数及运算中间结果；a b + c / f d e / - -。原中缀式为(a+b)/c-(f-d/e)。转换时操作数直接输出，运算符按优先级入出栈；求值时栈内存数值。

3．KMP失配后的模式字符（3分）

a。首次在主串第9位y与模式第9位a处失配。模式前8字符abceabce的最长相等真前后缀为abce，next[9]=5，主串指针不动，模式指针回退到第5位a。

4．二维数组行列地址（3分）

Loc(A)+64；Loc(A)+56。行优先偏移2×(4×7+4)=64字节；列优先偏移2×(4×6+4)=56字节。

5．Huffman译码与最小频率字符（3分）

GIHJG；I和J。图中F=00、G=01、H=10、I=110、J=111，码文分割为01 / 110 / 10 / 111 / 01。

I、J恰为仅有的两个最深叶。若它们中某个字符频率大于较浅叶的频率，交换两叶会减小WPL，违背Huffman树的最优性。频率互异时可确定I、J为最小二者，但不能仅据该图判定其中谁更小。

6．三维数组地址（3分）

0；162。A[3][4][5]是第120个元素，首地址为238-119×2=0；A[2][3][3]的元素偏移为2×5×6+3×6+3=81，地址为81×2=162。

7．先序与叶数（3分）

a,b,c,e,d；2个。根a的左孩子b、右孩子d；b只有右孩子c，c只有左孩子e，叶为e、d。

8．二叉树最小和最大高度（3分）

ceil(log_2(n+1))；n，n≥1。前者亦可写floor(log_2 n)+1；完全二叉树达到最小高度，单支链达到最大高度。

9．带哨兵顺序查找失败次数（3分）

n+1次。与n个实际记录的关键字比较均失败后，还须与存有待查关键字的哨兵比较一次，才能退出循环。

10．后缀求值的最小栈容量（3分）

3。表达式为6 4 2 - / 3 2 * +；各记号处理后栈深依次为1,2,3,2,1,2,3,2,1，最大值为3。

三、综合应用题

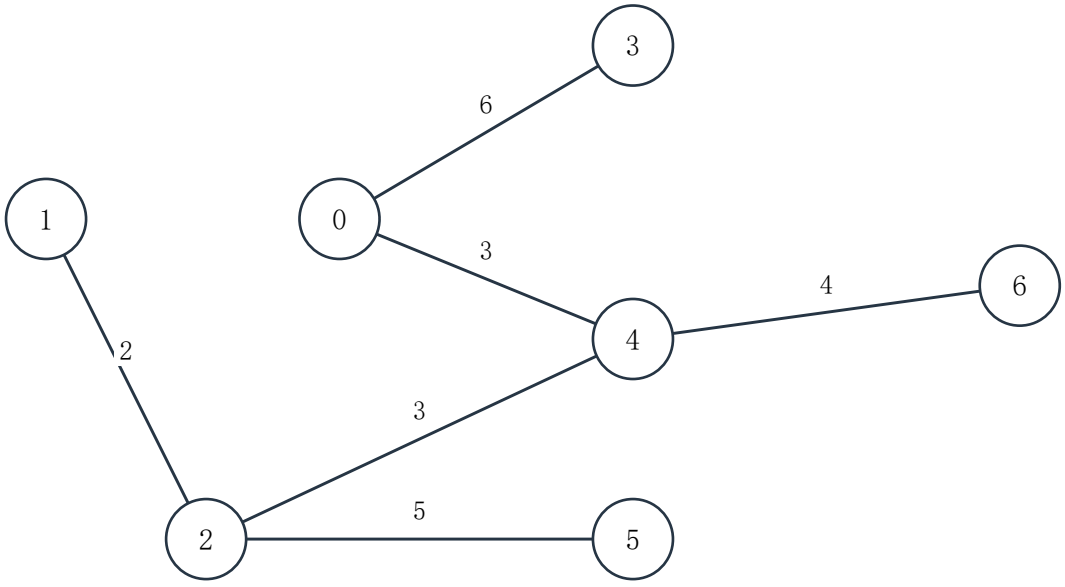
1．Kruskal最小代价生成树（10分）

（1）答案：一种最小生成树的边集为 {1-2，0-4，2-4，4-6，2-5，0-3}，总代价为 23。

计算过程：将边按权值非递减排列，依次选择连接不同连通分量的边；权值相同时按端点编号顺序考察。

考察的边	权值	处理	累计代价
1-2	2	选择	2
0-4	3	选择	5
2-4	3	选择	8
0-1	4	形成环，舍去	8
4-6	4	选择	12
2-5	5	选择	17
4-5	5	形成环，舍去	17
0-3	6	选择；已得到6条边，结束	23

总费用为2+3+3+4+5+6=23。由于2-5和4-5的权值均为5，将边2-5替换为4-5，也可得到另一棵代价为23的最小生成树。

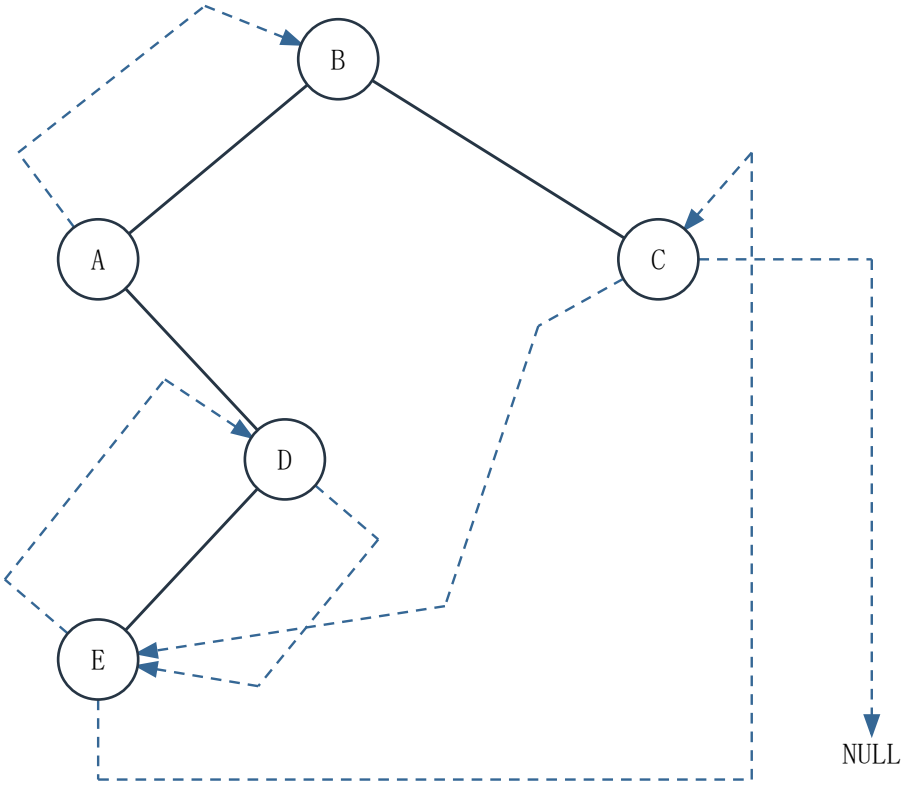


Kruskal所得一棵最小生成树，总代价23

（2）时间复杂度：将m条边排序需O(m log m)时间；使用并查集判环、合并的总时间为O(mα(n))，因此通常记Kruskal算法的时间复杂度为 **O(m log m)**。对简单无向图，因m≤n(n-1)/2，也常写为O(m log n)。

2．先序线索树、森林及双亲表示（10分）

（2）先序线索树：原二叉树先序为B,A,D,E,C。不设头结点；tag=0表示孩子指针，tag=1表示线索。原空左指针指向先序前驱，原空右指针指向先序后继。

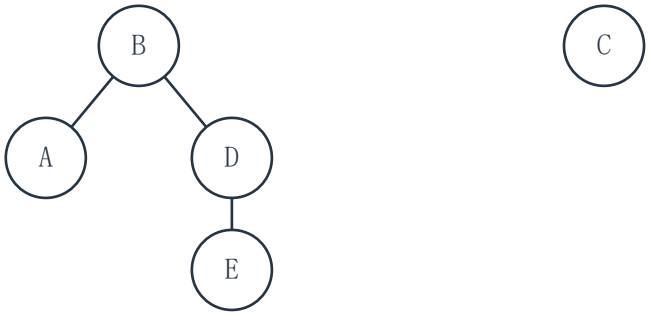


先序线索树：黑色实线为孩子边，蓝色虚线箭头为原空指针的线索

结点	Ltag	lchild指向	Rtag	rchild指向
B	0	A	0	C
A	1	B（前驱）	0	D
D	0	E	1	E（后继）
E	1	D（前驱）	1	C（后继）
C	1	E（前驱）	1	NULL

D的左孩子和右线索都指向E，但性质不同，不能只画一条不加说明的连接线。

（3）对应森林：按左孩子-右兄弟关系还原。根为B、C；B的孩子从左到右为A、D，D的孩子为E。



原二叉树对应的两棵有序树

（4）双亲表示法：一种合法存储如下，根的parent=-1；其他parent为双亲在本数组中的下标。

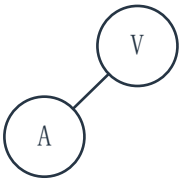
下标	data	parent
0	B	-1
1	A	0
2	D	0
3	E	2
4	C	-1

3．逐次构造AVL（10分）

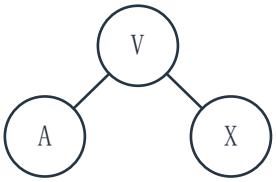
按字母顺序比较，平衡因子为左高减右高。以下每图均为插入该元素并完成必要调整后的状态。



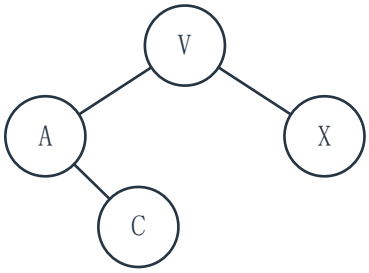
第1步：插入V后的AVL



第2步：插入A后的AVL

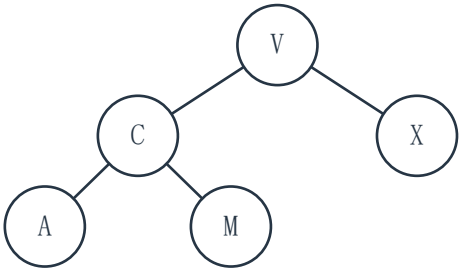


第3步：插入X后的AVL



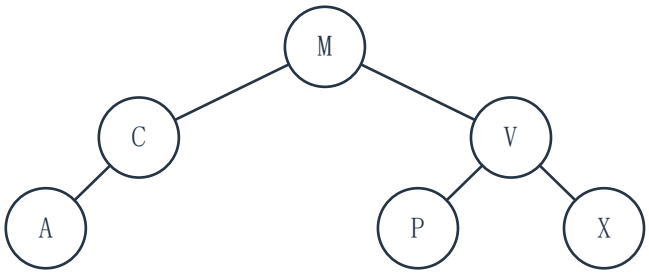
第4步：插入C后的AVL

前四步尚无失衡。插入M时，M先成为C的右孩子，最低失衡结点A为RR型，对A左旋：



第5步：插入M后的AVL

插入P时，P先成为M的右孩子，最低失衡结点V为LR型；先对C左旋，再对V右旋：



第6步：插入P后的AVL

最终根为M，左子树C的左孩子为A，右子树V的左右孩子为P、X。中序为A,C,M,P,V,X，所有结点左右高度差绝对值不超过1。

4．AOE事件时间与关键路径（10分）

题面缺漏：原图只标出“a9=8”，没有画出对应弧线和箭头，因而无法确定a9的弧尾与弧头。仅依据现有题面，**本题没有唯一的数值答案**。以下给出两种补图假设下的条件解，供核对原图；两种假设均不是题面已知条件。

原图能够确定的8条活动为：a1：0→2(2)，a2：0→1(3)，a3：1→2(4)，a4：1→3(4)，a5：2→4(3)，a6：4→3(2)，a7：4→5(3)，a8：3→5(6)。括号内为持续时间。

- **情况A：**假设缺失的a9为2→3，持续时间8。 - **情况B：**假设缺失的a9为1→4，持续时间8。

计算方法：取拓扑序0、1、2、4、3、5。前推时ve(v)=max{ve(u)+d(u,v)}；逆推时vl(u)=min{vl(v)-d(u,v)}。令源事件ve(0)=0，汇事件vl(5)=ve(5)。

事件	情况A：ve	情况A：vl	情况B：ve	情况B：vl
0	0	0	0	0
1	3	3	3	3
2	7	7	7	8
3	15	15	13	13
4	10	13	11	11
5	21	21	19	19

两种情况下的关键计算分别为：情况A中ve(3)=max(3+4,10+2,7+8)=15，ve(5)=max(10+3,15+6)=21；情况B中ve(4)=max(7+3,3+8)=11，ve(3)=max(3+4,11+2)=13，ve(5)=max(11+3,13+6)=19。

对活动a=(u,v)，最早开始时间e(a)=ve(u)，最晚开始时间l(a)=vl(v)-d(a)，时差为l(a)-e(a)。时差为0即为关键活动。

活动	A：e	A：l	A：时差	B：e	B：l	B：时差
a1	0	5	5	0	6	6
a2	0	0	0	0	0	0
a3	3	3	0	3	4	1
a4	3	11	8	3	9	6
a5	7	10	3	7	8	1
a6	10	13	3	11	11	0
a7	10	18	8	11	16	5
a8	15	15	0	13	13	0
a9	7	7	0	3	3	0

补图假设	关键活动	关键路径	路径长度及工期
A：a9为2→3(8)	a2、a3、a9、a8	0→1→2→3→5	3+4+8+6=21
B：a9为1→4(8)	a2、a9、a6、a8	0→1→4→3→5	3+8+2+6=19

5．线性探测散列表与查找命中ASL（10分）

答案：最终散列表如下，等概率查找命中的平均查找长度为 ASL=2。

地址	0	1	2	3	4	5	6	7	8	9
关键字	空	15	空	17	11	10	13	12	19	空
成功比较次数	-	1	-	1	1	3	1	3	4	-

计算过程：初始散列地址取H(key)=key mod7；线性探测时应在整个长度为10的表内移动，故H_i(key)=[H(key)+i] mod10。取模7只用于计算初始地址，不能将后续探测也限制在0-6。

关键字	初始地址key mod7	依次探查地址	最终地址	成功比较次数
11	4	4	4	1
13	6	6	6	1
15	1	1	1	1
17	3	3	3	1
10	3	3→4→5	5	3
12	5	5→6→7	7	3
19	5	5→6→7→8	8	4

比较次数包含在初始地址处的第一次关键字比较。因此，ASL成功=(1+1+1+1+3+3+4)/7=14/7=2。

6．判断小根堆（10分）

答案：两组序列**都已是小根堆，无需调整**。小根堆只要求每个父结点的值不大于其孩子，允许关键字相等。

采用1起始下标，依次检查所有有孩子的结点。

（1）序列{12, 24, 33, 65, 33, 56, 48, 92, 86, 70}：

父结点下标i	父结点值	左孩子	右孩子	检查结果
1	12	24	33	12≤24且12≤33
2	24	65	33	24≤65且24≤33
3	33	56	48	33≤56且33≤48
4	65	92	86	65≤92且65≤86
5	33	70	无	33≤70

（2）序列{05, 23, 20, 28, 40, 38, 29, 61, 35, 76, 47, 100}：

父结点下标i	父结点值	左孩子	右孩子	检查结果
1	5	23	20	5≤23且5≤20
2	23	28	40	23≤28且23≤40
3	20	38	29	20≤38且20≤29
4	28	61	35	28≤61且28≤35
5	40	76	47	40≤76且40≤47
6	38	100	无	38≤100

两组序列的全部父子关系均满足小根堆条件，因此保留原序列即可。

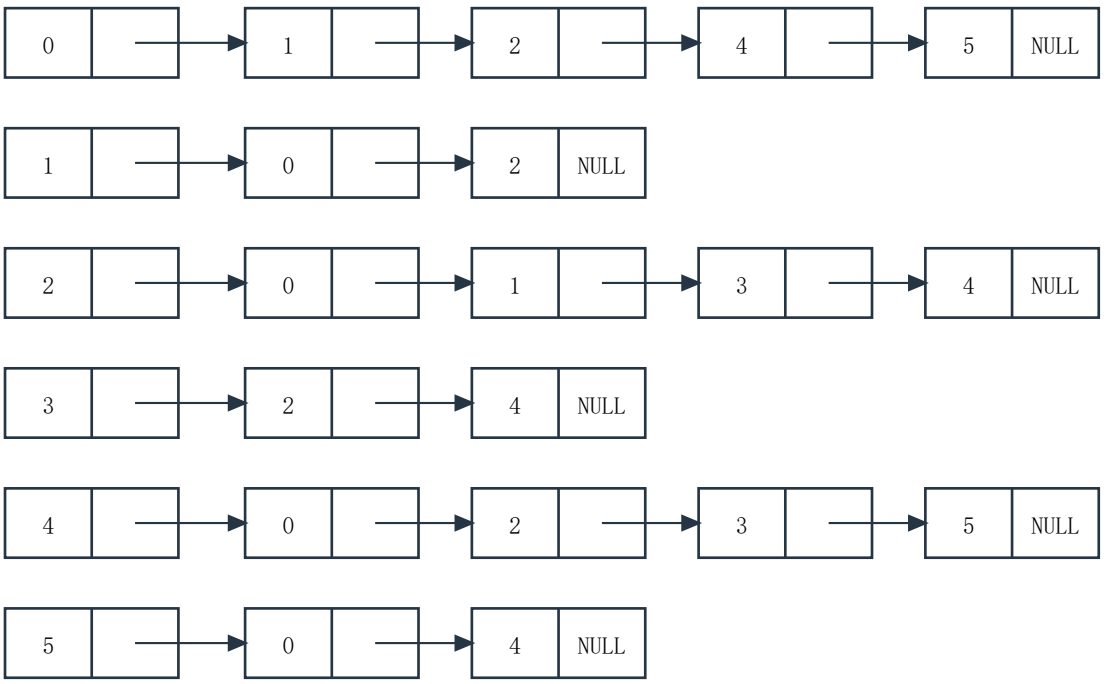
7．邻接矩阵、邻接表与图遍历（10分）

（1）**邻接矩阵**：行、列均按顶点0、1、2、3、4、5排列；有边记1，无边记0。

顶点	0	1	2	3	4	5
0	0	1	1	0	1	1
1	1	0	1	0	0	0
2	1	1	0	1	1	0
3	0	0	1	0	1	0
4	1	0	1	1	0	1
5	1	0	0	0	1	0

邻接表：按题定边的输入次序，对每条无向边在两个端点的边表中分别尾插。因此，各顶点的邻接点顺序如下，遍历时按表中从左到右的次序扫描。

顶点	邻接链表
0	1→2→4→5→NULL
1	0→2→NULL
2	0→1→3→4→NULL
3	2→4→NULL
4	0→2→3→5→NULL
5	0→4→NULL



邻接表：左栏为顶点及首边指针，链结点为邻接点及next指针

（2）深度优先遍历：从顶点3出发，序列为 **3，2，0，1，4，5**。

对应DFS树边为{3-2，2-0，0-1，0-4，4-5}。

广度优先遍历：从顶点3出发，在顶点首次入队时标记已访问，序列为 **3，2，4，0，1，5**。

对应BFS树边为{3-2，3-4，2-0，2-1，4-5}。若邻接表采用其他合法邻接次序，遍历序列可能不同，须与所画邻接表保持一致。

四、算法设计与分析题

1．连接两个带头结点单链表（10分）

基本思想：找到X的尾结点，将其link接到Y的首个数据结点。Y的头结点不应接入X的数据链；本实现保留Y的头结点，并把Y置为空表，避免两表头同时拥有同一条数据链。假定X、Y为互不共享结点的两张合法链表。

C语言实现：

```
#include <stddef.h>
typedef struct Node {
    int data;
    struct Node *link;
} Node;
void concatenate(Node *X, Node *Y) {
    Node *p = X;
    while (p->link != NULL) p = p->link;
    p->link = Y->link;
    Y->link = NULL;
}
```

复杂度：设X原有m个数据结点，时间O(m)，仅使用一个游标，辅助空间O(1)。X或Y为空表均适用。若约定连接后不再保留Y的头结点，也可在接链后释放Y头，但不能把该头结点作为数据结点接入结果。

2．单链表的简单选择排序（10分）

基本思想：将表分成已排序前段和未排序后段。p依次指向后段首结点，每趟从p开始扫描，找最小值结点min，与p交换数据，再将p向后移动一位。原头结点和所有link指针保持不变。

C语言实现：

```
#include <stddef.h>
typedef struct Node {
    int data;
    struct Node *link;
} Node;
void selectionSort(Node *L) {
    for (Node *p = L->link; p != NULL; p = p->link) {
        Node *min = p;
        for (Node *q = p->link; q != NULL; q = q->link)
            if (q->data < min->data) min = q;
        if (min != p) {
            int temp = p->data;
            p->data = min->data;
            min->data = temp;
        }
    }
}
```

复杂度：比较次数为(n-1)+(n-2)+...+1=n(n-1)/2，时间O(n^2)，辅助空间O(1)。空表、单结点表均正确。此交换数据的简单选择排序一般不稳定；本题未要求稳定性，也未禁止交换数据。

3．以非增次序打印BST，常数辅助空间（10分）

基本思想：BST的逆中序“右-根-左”给出非增序列。为满足只使用若干指针变量的限制，采用逆中序Morris遍历，利用右子树最左结点原本为空的lchild暂存返回线索；第二次沿线索回到当前结点时立即清除，再访问当前结点。无需递归或显式栈。

C语言实现：

```
#include <stdio.h>
#include <stddef.h>
typedef struct TNode {
    int data;
    struct TNode *lchild, *rchild;
} TNode;
void printNonIncreasing(TNode *T) {
    TNode *cur = T, *p;
    while (cur != NULL) {
        if (cur->rchild == NULL) {
            printf("%d ", cur->data);
            cur = cur->lchild;
        } else {
            p = cur->rchild;
            while (p->lchild != NULL && p->lchild != cur)
                p = p->lchild;
            if (p->lchild == NULL) {
                p->lchild = cur;
                cur = cur->rchild;
            } else {
                p->lchild = NULL;
                printf("%d ", cur->data);
                cur = cur->lchild;
            }
        }
    }
}
```

复杂度：每条相关边仅被常数次扫描，总时间O(n)，只用固定个数指针，辅助空间O(1)。所有临时线索在正常遍历结束时恢复为NULL，原树结构保持原样。普通递归逆中序会使用O(h)调用栈，不能严格满足本题的空间限制。